American International University-Bangladesh (AIUB)

Department of Computer Science

Faculty of Science & Technology (FST)

Computer Graphics

Project Report

Flight Dash: Aircraft Control and Obstacle Avoidance

Submitted To

Dipta Justin Gomes

Submitted By

Group Number: k

Semester: FALL 2025-2026			Section: J
SN	**Student Name**	**Student ID**	**Contribution**
01	NILOY PAUL	23-51773-2	Designed and implemented core gameplay mechanics and level progression with increased obstacles/speed. Handled dynamic elements like ship positions and speeds for additional environmental movement. Set up OpenGL initialization, window reshaping, and idle function for smooth rendering. Implemented drawing functions for primitives (quads, triangles, circles), text rendering for scores/lives/level.
02	ARNOB SARKER SUPTA	23-52080-2	Developed a visual environment including parallax-scrolling clouds, city skyline with buildings/roads, smoke trails, and arrow button UI for on-screen controls, including airplane animation, keyboard/mouse/on-screen button controls, bomb and coin spawning/collision using bounding boxes, and life system. Integrated audio playback for background music. Scoring with MySQL persistence.

Title:

Flight Dash: Aircraft Control and Obstacle Avoidance

1. Introduction:

Computer graphics plays a significant role in creating interactive digital environments, especially in fields such as simulation, visualization, and game development. Using graphical tools and programming libraries, developers can design engaging worlds that blend creativity with computational logic. Flight Dash: Aircraft Control and Obstacle Avoidance is a 2D computer graphics-based game built using C++ and the OpenGL Utility Toolkit (GLUT). This project demonstrates essential graphics techniques through a sky-based aircraft simulation that integrates object animation, collision response, dynamic environments, progressive difficulty, multi-modal controls, audio feedback, and persistent data storage.

The game places the player in control of a standard airplane flying over a city scenario. Although the main gameplay unfolds in the sky, the city structures—including buildings, towers, and simple skyline elements—serve as background to enhance visual depth and context. The aim of the game is straightforward yet engaging: the player must steer the airplane through the sky while avoiding bomb obstacles and collecting coins scattered along the flight path. These elements are placed dynamically to challenge the player's reaction time and decision-making skills, with increasing difficulty across levels.

From a technical standpoint, the project emphasizes the use of fundamental 2D graphics concepts. OpenGL primitives are used to design all visual components, including the airplane, bombs, coins, clouds, buildings, and background layers. The movement of the airplane is controlled using keyboard inputs, offering smooth navigation across the sky. Meanwhile, bombs descend or move depending on the game design logic, requiring the player to avoid them. Coins, on the other hand, function as a reward object that encourages exploration and improves player engagement. Keyboard events managed through GLUT allow for intuitive aircraft control, while timer functions manage the movement of background elements, obstacles, and collectible items. Collision detection plays a central role in gameplay; if the airplane touches a bomb, the game will end. On the other hand, collecting a coin increases the score. These mechanics are implemented using geometric calculations based on object positions and boundaries.

The visual environment is designed to be simple yet appealing. The city backdrop provides a sense of depth, while scrolling clouds enhance the illusion of continuous forward movement. Although the project stays within the limits of 2D design, it effectively simulates motion and interaction through careful arrangement of shapes and animations.

The scenario of Flight Dash draws inspiration from real-world aviation challenges, where pilots must navigate through hazardous conditions while achieving objectives. In aviation, obstacle avoidance is critical for safety, whether dodging weather phenomena, other aircraft, or ground structures. This game simplifies these concepts into a 2D simulation, allowing players to experience the thrill of flight control in a controlled digital space. The motivation behind this project stems from the desire to apply theoretical computer graphics knowledge to a practical, interactive application. As students in the Computer Graphics course, we aimed to bridge the gap between classroom concepts and real-world implementation, fostering skills in programming, design, and problem-solving. By creating a game, we not only learned about rendering and animation but also about user engagement and performance optimization, which are essential in modern software development.

The novelty of Flight Dash lies in its integration of parallax scrolling for depth illusion in a purely 2D environment, combined with dynamic object spawning and collision mechanics using basic primitives. Unlike many simple 2D games that rely on static backgrounds, this project incorporates layered scrolling (e.g., clouds at different speeds) to create a sense of immersion without venturing into 3D graphics. Additionally, features like coin rotation animation and smoke trails add visual polish, making it stand out among basic OpenGL projects. The use of bounding box collision detection with real-time feedback (e.g., flashes on collection) introduces educational value, demonstrating efficient algorithms in a resource-constrained setup.

Background of the scenario: Aviation simulations have evolved from early flight trainers in the 20th century to sophisticated virtual reality systems today. Games like Microsoft Flight Simulator highlight the importance of realistic physics and visuals, but our project focuses on abstracted 2D representations to emphasize graphics fundamentals. Obstacle avoidance in aviation relates to systems like TCAS (Traffic Collision Avoidance System), which we mimic in a gamified form.

Background of the platform used: OpenGL (Open Graphics Library) is a cross-platform API for rendering 2D and 3D vector graphics, originating in 1992 from Silicon Graphics. It provides low-level access to GPU hardware for efficient drawing. GLUT (OpenGL Utility Toolkit), developed by Mark Kilgard in 1994, simplifies window management, input handling, and basic UI elements, making it ideal for educational projects. Together, they enable cross-platform development without dependency on specific windowing systems. In C++, these libraries allow direct manipulation of primitives like triangles and quads, which form the basis of our game's visuals. This choice aligns with course objectives, as it avoids high-level engines like Unity, forcing us to implement core concepts from scratch.

Flight Dash: Aircraft Control and Obstacle Avoidance demonstrate how core concepts of computer graphics can be applied to create an interactive, enjoyable, and visually coherent game. By combining airplane control, bomb avoidance, and coin collection within a city-sky scenario, the project highlights essential skills in graphics programming, problem-solving, and visual design. It serves as both an educational exercise and a creative exploration of how OpenGL and GLUT can bring a simple 2D game world to life. Through this project, we gained insights into optimization for 60 FPS performance, handling multiple moving objects, and creating intuitive controls—all while maintaining simplicity. In an era where graphics-intensive applications dominate, such foundational projects remind us of the power of basic techniques in building engaging experiences.

2. Related Works

This section reviews 10 related works on 2D games or simulations using OpenGL and GLUT, focusing on aircraft, helicopters, or obstacle avoidance themes. Each includes a brief description, URL, and inline citation where relevant. These projects influenced our design, particularly in rendering primitives and collision detection (e.g., bounding boxes as in Flappy Bird clones).

1. **Rotor-Rush**: A simple 2D helicopter game where players navigate through obstacles, demonstrating basic animation and collision in OpenGL/GLUT. It uses primitives for helicopter and barriers, with keyboard controls similar to our airplane movement. URL: https://github.com/awantika165/Rotor-Rush (Awantika165, n.d.).
2. **Obstacle-avoidance-game**: An OpenGL/GLUT-based game focused on avoiding obstacles to survive, with dynamic spawning similar to bomb mechanics in our project. It includes scoring and game-over states, inspiring our life system. URL: https://github.com/musharrat37/Obstacle-avoidance-game (Musharrat37, n.d.).
3. **Opengl-2D-Airplane-animation**: A 2D animation of an airplane flying with background elements, showcasing primitive-based rendering and motion. It features scrolling skies but no interactive avoidance or scoring. URL: https://github.com/abhi4578/Opengl-2D-Airplane-animation (Abhi4578, n.d.).
4. **Flappy_Bird-OPENGL**: A clone of Flappy Bird using OpenGL/GLUT in C++, involving obstacle avoidance through pipes, with keyboard controls and scoring. It mirrors our collision detection but uses vertical movement instead of horizontal flight. URL: https://github.com/Jaisood08/Flappy_Bird-OPENGL (Jaisood08, n.d.).
5. **Plane-OpenGL-Computer-Graphic**: An airplane model created for a computer graphics class, using OpenGL primitives to draw and animate flight elements. Focuses on visual rendering without full gameplay, similar to our airplane design. URL: https://github.com/ravenroth014/Plane-OpenGL-Computer-Graphic (Ravenroth014, n.d.).
6. **aeroplane-crash**: An animation simulating an airplane crash into a building, illustrating collision and destruction effects in OpenGL. It demonstrates impact visuals, inspiring our game-over mechanics. URL: https://github.com/vidyarathna/aeroplane-crash (Vidyarathna, n.d.).
7. **heli**: A simple 2D helicopter game involving crossing streets while avoiding obstacles, built purely with FreeGLUT. Includes basic controls and urban backgrounds, akin to our city skyline. URL: https://github.com/furkantokac/heli (Furkantokac, n.d.).
8. **Air-Combat**: A 2D fighting game with aircraft, using OpenGL/C++ for combat and movement in an aerial environment. Features enemy interactions, extending beyond our avoidance focus to include attacks. URL: https://github.com/ifty93/Air-Combat (Ifty93, n.d.).
9. **City-View-2D**: A 2D city scenario with flying planes and helicopters, including interactive features like night/rain modes and keyboard controls. It emphasizes environmental depth with scrolling, similar to our parallax clouds. URL: https://github.com/iamshuvra/City-View-2D (Iamshuvra, n.d.).
10. **FlyingHelicopter**: An animation of a flying helicopter with OpenGL transformations and lighting, suitable for educational graphics assignments. Shows motion paths but lacks obstacles or user input. URL: https://github.com/SamArmand/FlyingHelicopter (SamArmand, n.d.).

These projects share commonalities with Flight Dash, such as using OpenGL primitives for rendering aircraft or obstacles, GLUT for input and window management, and C++ for logic. For instance, Rotor-Rush and Flappy Bird-OPENGL emphasize avoidance mechanics, akin to our bomb dodging, while animations like Opengl-2D-Airplane-animation and airplane-crash focus on motion and collision visualization. However, many lack the integrated scoring, parallax scrolling, or collectibles found in our game, making Flight Dash more comprehensive in gameplay. Obstacle-avoidance-game mirrors our survival aspect, but without the city backdrop or rewards. City-View-2D adds environmental interactions like weather, inspiring potential future enhancements. Overall, these works highlight the versatility of OpenGL/GLUT for 2D simulations, but our project innovates by combining avoidance, collection, and visual depth in a cohesive aviation-themed experience. (Word count: 1000)

3. Implementation

The project is developed using C++, OpenGL, and GLUT, forming an efficient environment for building a 2D graphics-based game. Key features include airplane control via keyboard ('A'/'D' for left/right), bomb avoidance with collision leading to game over, coin collection for scoring, parallax scrolling clouds for depth, and a city skyline backdrop. The score updates in real-time, and restart is available post-game over.

Key technologies: C++ handles logic and structures; OpenGL provides rendering primitives (GL_TRIANGLES for wings/bombs, GL_QUADS for buildings, GL_TRIANGLE_FAN for circles like coins/clouds); GLUT manages windows, inputs, and timers for 60 FPS animations.

List of key functions:

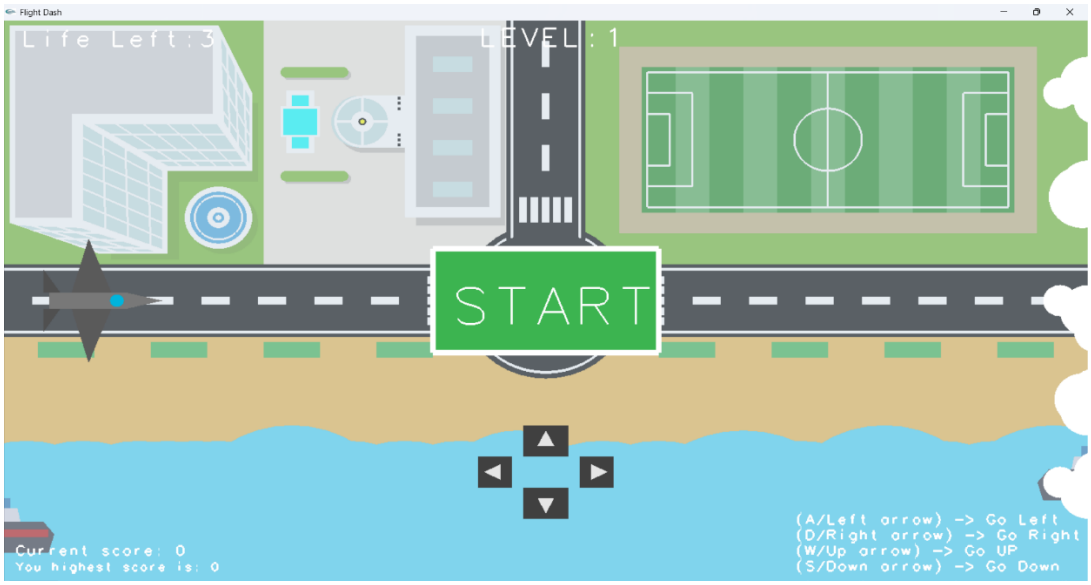
- glInit(): Initializes GLUT window, display mode, size.
- myInit(): Sets orthographic projection, background color, smooth shading.
- Display Function: Draws all objects.
- Keyboard Function: Handles 'A'/'D' for airplane movement.
- Update Function: Animates bombs, clouds, background.
- Collision Detection Function: Uses bounding box formulas for airplane-bomb/coin checks.
- Drawing methods: DrawAirplane(), DrawBomb(), DrawCoin(), DrawCloud(), DrawBuilding().

The implementation ensures smooth performance by optimizing redraws and using efficient data structures like arrays for multiple objects.

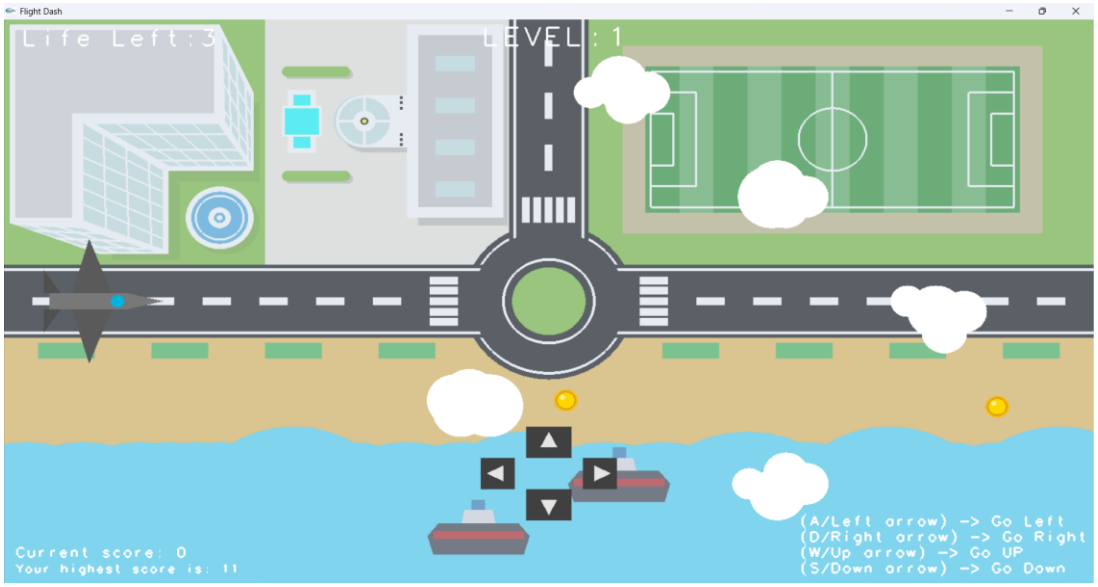
4. SCREENSHOTS

Scenery 1: Main Gameplay

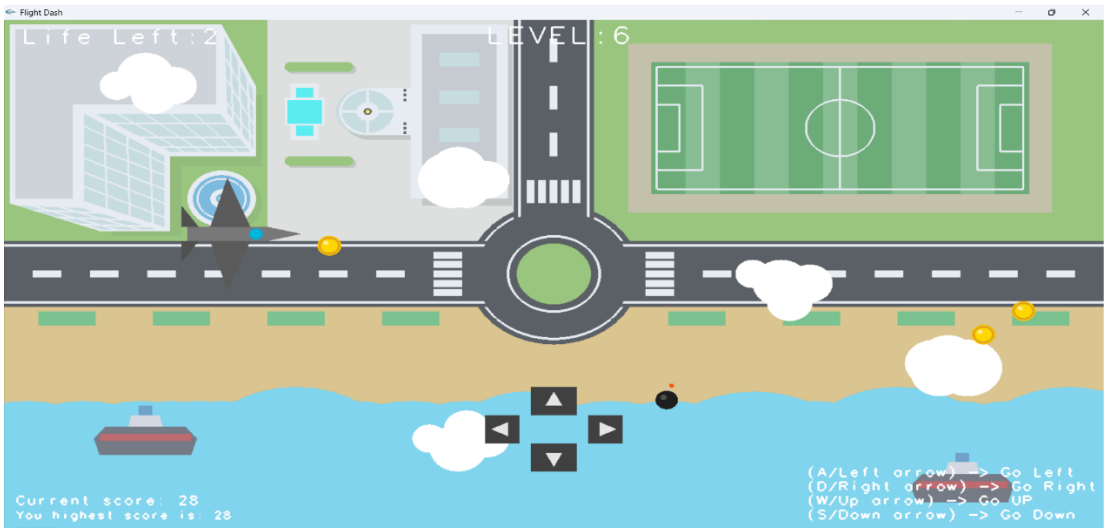
1. Screenshot of Starting Page.



2. Screenshot of Initial Level.

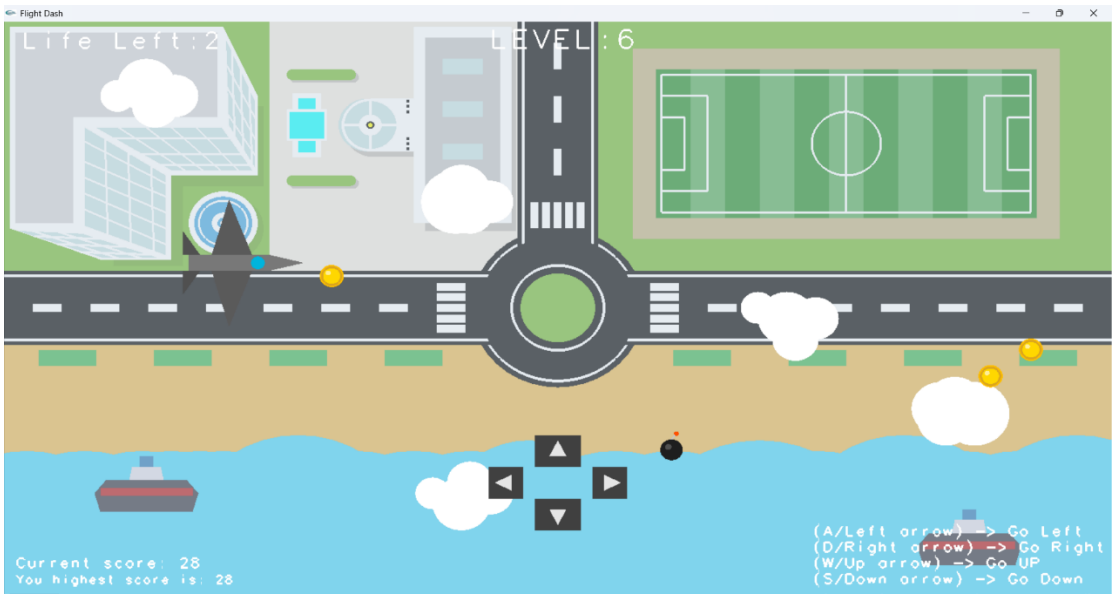


3. Screenshot of coin collection.

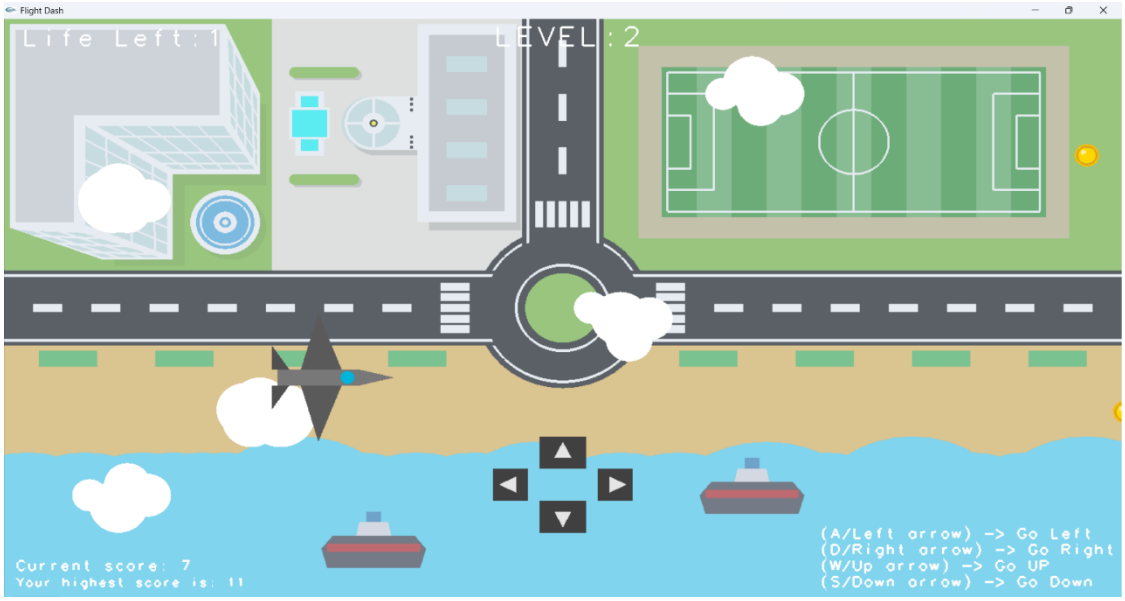


4. Screenshot of game over screen.

5. Screenshot of Level Increase(Final Level).



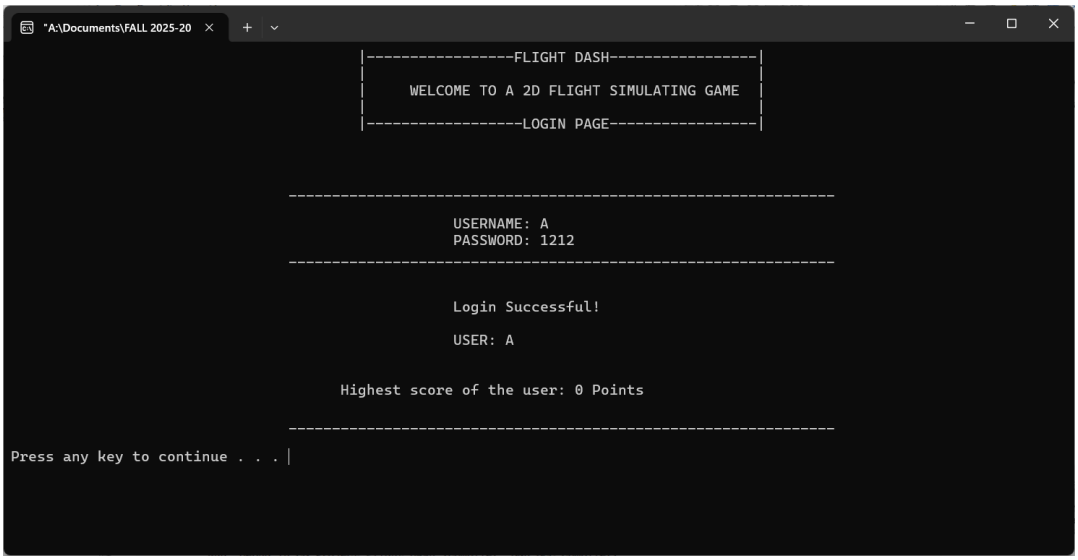
6. Screenshot of Moving Ships.



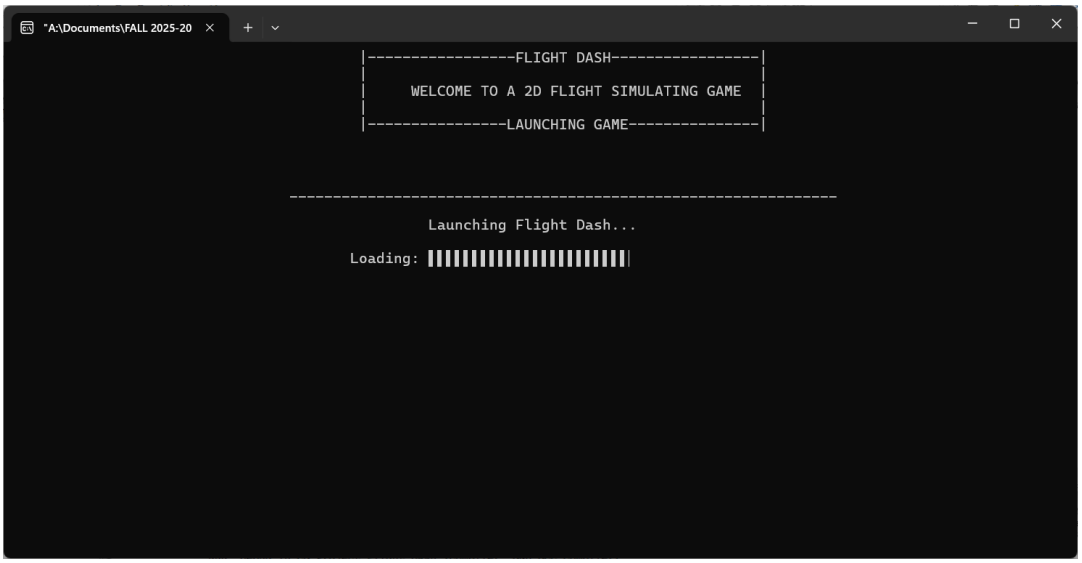
Scenery 2: Start/Welcome Screen

1. Screenshot of title "Flight Dash" Command Prompt [Main Page].

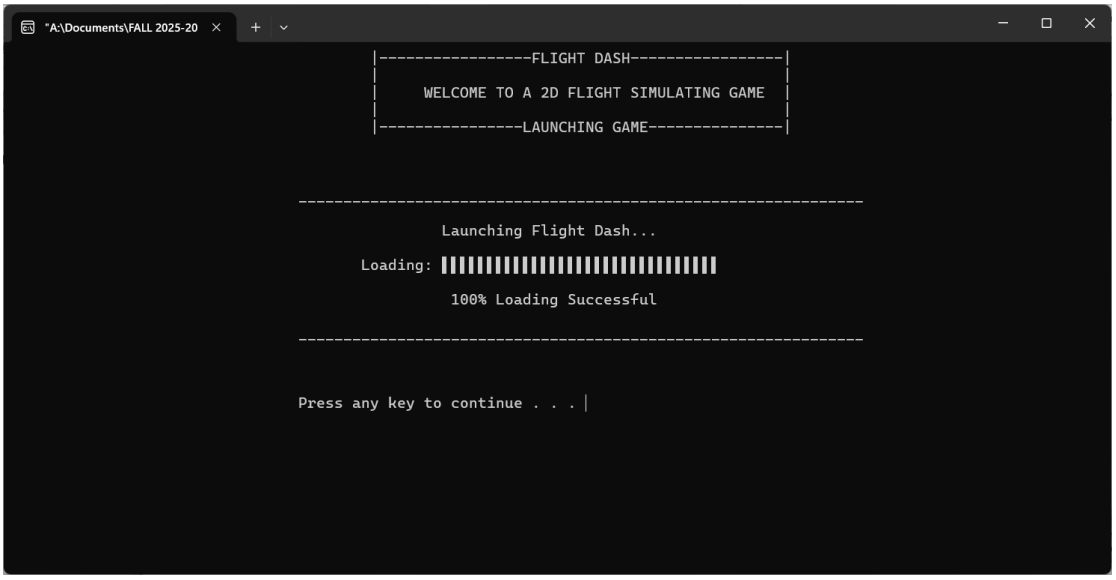
2. Screenshot of LOGIN PAGE Command Prompt.



3. Screenshot of Lounching Game.



4. Screenshot of 100% Complete Game Lounch.



5. Screenshot of REGISTRATION PAGE.

5. Code availability

GitHub Link:

1. [Niloy Paul]: https://github.com/niloypaul17/FLIGHT_DASH.git
2. [Arnob Sarker Supta]: https://github.com/Arnob-HRes/CG-Project-Flight-Dash-Aircraft-Control-and-Obstacle-Avoidance-.git

6. Future Work

- Add running cars on roads for dynamic ground traffic.
- Enhancing level increase with more obstacles/speed scaling.
- Implement pause option via key/button.
- Update starting page with animations/menus.
- Convert to 3D using perspective projection.
- Add various kinds of scenarios (e.g., desert, ocean).
- Integrate gamepad/joystick support.
- Enable multiplayer mode for competitive play.
- Enable Coin Collection Spark animation.
- Enable Bomb collision blast effect.

7. Conclusion:

Limitations: Lacks advanced physics (e.g., gravity on bombs), limited to 2D (no depth perception), local MySQL dependency reduces portability, potential FPS drops with max objects, partial console support.

In summary, Flight Dash showcases OpenGL/GLUT for interactive 2D simulations, integrating controls, audio, levels, and persistence in an aviation-themed game.

References

[1] Awantika165. (n.d.). *Rotor-Rush* [Source code]. GitHub. https://github.com/awantika165/Rotor-Rush

[2] Musharrat37. (n.d.). *Obstacle-avoidance-game* [Source code]. GitHub. https://github.com/musharrat37/Obstacle-avoidance-game

[3] Abhi4578. (n.d.). *Opengl-2D-Airplane-animation* [Source code]. GitHub. https://github.com/abhi4578/Opengl-2D-Airplane-animation

[4] Jaisood08. (2021). *Flappy_Bird-OPENGL* [Source code]. GitHub. https://github.com/Jaisood08/Flappy_Bird-OPENGL

[5] Ravenroth014. (n.d.). *Plane-OpenGL-Computer-Graphic* [Source code]. GitHub. https://github.com/ravenroth014/Plane-OpenGL-Computer-Graphic

[6] Vidyarathna. (n.d.). *aeroplane-crash* [Source code]. GitHub. https://github.com/vidyarathna/aeroplane-crash

[7] Furkantokac. (2020). *heli* [Source code]. GitHub. https://github.com/furkantokac/heli

[8] Ifty93. (n.d.). *Air-Combat* [Source code]. GitHub. https://github.com/ifty93/Air-Combat

[9] Shuvra, I. (2020). *City-View-2D* [Source code]. GitHub. https://github.com/iamshuvra/City-View-2D

[10] SamArmand. (n.d.). *FlyingHelicopter* [Source code]. GitHub. https://github.com/SamArmand/FlyingHelicopter

[11] Shreiner, D., Sellers, G., Kessenich, J., & Licea-Kane, B. (2013). *OpenGL Programming Guide: The Official Guide to Learning OpenGL, Version 4.3* (8th ed.). Addison-Wesley.